



Class: SE

Subject: DBMS

Sem-IV

Experiment No: 6

Course Outcome: Write and execute complex and nested SQL queries using SQL commands

Blooms Level: Level 3

Aim: Perform Nested and Complex queries.

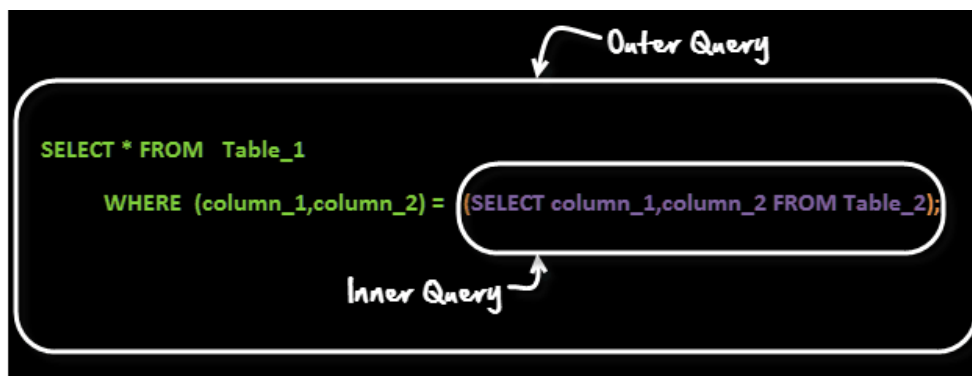
Abstract:

This experiment focuses on the use of complex and nested SQL queries to retrieve meaningful information from relational databases. It involves applying subqueries and multi-level complex queries along with filtering, grouping, and aggregation operations. The experiment enhances understanding of advanced SQL querying techniques for efficient data analysis and retrieval.

Theory:

SQL provides powerful querying capabilities to retrieve data from one or more tables in a relational database. Complex queries involve the use of multiple SQL clauses such as JOIN, GROUP BY, HAVING, and aggregate functions to extract meaningful information from large datasets.

Nested Query :



A nested query (subquery) is a query written inside another SQL query. The result of the inner query is used by the outer query for further processing. Subqueries can be classified as single-row, multiple-row, and correlated subqueries. Nested queries are commonly used for comparison, filtering, and existence checks using operators such as IN, ANY, ALL, and EXISTS.

Correlated subqueries depend on values from the outer query and are evaluated repeatedly for each row processed by the outer query. These queries are useful for row-wise comparisons and advanced data analysis.

Complex Queries :

Complex queries are SQL queries that involve multiple operations such as joins, nested queries, grouping, aggregation, and conditional filtering to retrieve meaningful information from one or more tables. These queries use clauses like JOIN, WHERE, GROUP BY, HAVING, and aggregate functions to perform advanced data analysis. Complex queries help in efficiently extracting and summarizing data from large relational databases and are widely used in reporting and decision-making applications.

Output:

Some nested and complex queries are written on following schema

Sailors (SID, Sname, Rating, Age)

Boat (BID, Bname , Colour)

Reserved (SID, BID, Date)

```
mysql> select * from sailors;
+-----+-----+-----+-----+
| sid | sname  | rating | age  |
+-----+-----+-----+-----+
| 22  | dustin | 7      | 45.00 |
| 29  | brutus | 1      | 33.00 |
| 31  | lubber | 8      | 55.50 |
| 33  | andy   | 8      | 25.50 |
| 58  | rusty  | 10     | 35.00 |
| 64  | horatio | 7      | 35.00 |
| 71  | zorba  | 10     | 16.00 |
| 74  | horatio | 9      | 35.50 |
| 85  | art    | 3      | 25.50 |
| 95  | bob    | 3      | 63.50 |
+-----+-----+-----+-----+
10 rows in set (0.01 sec)
```

```
mysql> select * from reserves;
+-----+-----+-----+
| sid | bid | day       |
+-----+-----+-----+
| 22  | 101 | 1998-10-10 |
| 22  | 102 | 1998-10-10 |
| 22  | 103 | 1998-10-08 |
| 22  | 104 | 1998-10-07 |
| 31  | 102 | 1998-10-11 |
| 31  | 103 | 1998-06-11 |
| 31  | 104 | 1998-12-11 |
| 64  | 101 | 1998-05-09 |
| 64  | 102 | 1998-08-09 |
| 74  | 103 | 1998-08-09 |
+-----+-----+-----+
10 rows in set (0.01 sec)
```

```
mysql> select * from boat;
+-----+-----+-----+
| bid | bname   | colour |
+-----+-----+-----+
| 101 | interlake | blue  |
| 102 | interlake | red   |
| 103 | clipper  | green |
| 104 | marine   | red   |
+-----+-----+-----+
4 rows in set (0.01 sec)
```

Some sample nested and complex queries

- 1) Find the names of sailors who have reserved a red boat.

SQL Query : select s.sid,s.sname from sailors s where sid in(select sid from reserves where bid in (select bid from boat where colour='red'));

- 2) Find sailors whose rating is better than some sailor called Horatio.

SQL Query : select sid,sname,rating from sailors where rating > any (select rating from sailors where sname='horatio');

- 3) Find the sailor with the highest rating.

SQL Query : select sid,sname,rating from sailors where rating in(select max(rating) from sailors);

- 4) Find the name and age of the oldest sailor.

SQL Query : select s.sname, s.age from sailors s where age =(select max(age) from sailors);

- 5) Find the average age of sailors for each rating level that has at least two sailors

SQL Query : select rating,avg(age),count(*) from sailors group by rating having count(*)>1;

- 6) Find the names of sailors with a higher rating than all sailors with age < 21.

SQL Query : select * from sailors where rating > any (select rating from sailors where age<21);

Conclusion:

This experiment enhanced understanding of complex and nested SQL queries used for effective data retrieval and analysis. It demonstrated how subqueries and advanced SQL clauses help extract meaningful information from relational databases.

Exercise 1 :

Perform complex and nested SQL queries on the Management System database designed in Experiment No. 1.

Write and execute nested queries using IN, ANY, ALL to retrieve meaningful information from the database.

Additionally, formulate complex SQL queries involving JOIN, WHERE, GROUP BY, and HAVING clauses along with aggregate functions (COUNT, SUM, AVG, MIN, MAX) to analyze and compare data across multiple relations.

Exercise 2 :

Consider the schema

Employee(EID, Ename, Dept, Salary)

Project(PID, Pname, Dept)

Works_On(EID, PID, Hours)

- 1) Find the names of employees who earn more than the **average salary** of all employees.
- 2) Find the names of employees who work on projects belonging to the **‘Production’ department**.
- 3) Find the **project IDs** on which **more than two employees** are working.
- 4) Find the **department(s)** whose **average employee salary** is greater than the **overall average salary**.
- 5) Find the employees who earn more than **all employees** working in the **‘Finance’ department**.